



Universae

MESERO APP

Desarrollo de Aplicaciones Multiplataforma
Curso 2025/2026

Presentado por:
Roberto de Frutos
Jiménez

Índice

Índice.....	1
Introducción	2
Objetivos del proyecto.....	3
Objetivos generales:.....	4
Objetivos específicos:	4
Alcance del proyecto.....	4
Tecnologías utilizadas	4
Lenguaje de programación	4
Entorno de desarrollo.....	5
Arquitectura y patrones	5
Interfaz de usuario	5
Persistencia de datos.....	5
Comunicación.....	5
Prototipado y control de versiones	5
Evidencias de uso de tecnologías	6
Análisis del sistema.....	8
Implementación.....	8
Manual de usuario.....	8
Pruebas de caja blanca realizadas:.....	8
Prueba 1: Creación de bar con código incorrecto	9
Prueba 2: Registro de usuarios con correo duplicado	9
Prueba 3: Gestión de login	10
Prueba 4: Gestión de mesas y productos	11
Prueba 5: Error al acceder a comandas	13
Prueba 6: Navegación según rol de usuario	13
Prueba 7: Visualización del código del bar	14

Introducción

MeseroApp es una aplicación Android desarrollada en Java con Android Studio, orientada a la gestión completa de bares y sus comandas en tiempo real. Permite a los usuarios:

- Gestionar bares, mesas y productos.
- Crear y administrar pedidos/comandas por mesa.
- Controlar stock de productos.
- Enviar facturas por correo electrónico.
- Recibir notificaciones de pedidos listos.
- Mantener persistencia de datos con Room y SharedPreferences.
- Actualización automática de listas con LiveData.

La aplicación está pensada para ser modular, escalable y de fácil uso, combinando Frontend y Backend desde el diseño de UI hasta la lógica de negocio.

Enlace a él GitHub con el código fuente del proyecto: [MESERO APP](#)



Objetivos del proyecto

Objetivos generales:

Desarrollar una aplicación Android que funcione a la perfección para gestionar bares, facilitando el control de mesas, pedidos y productos de manera eficiente y en tiempo real.

Objetivos específicos:

- Crear un sistema para gestionar mesas y pedidos de manera eficiente.
- Desarrollar un control de inventario para los productos.
- Implementar la persistencia de datos utilizando Room y SharedPreferences.
- Enviar facturas por correo electrónico a través de SMTP.
- Configurar notificaciones para alertar sobre pedidos listos.
- Diseñar una interfaz amigable siguiendo los prototipos en Figma.

Alcance del proyecto

En este apartado se describen las funcionalidades que ofrece MESERO APP:

- Gestión de mesas: abrir/crear órdenes, cerrar órdenes, estados disponible/ocupada.
- Gestión de productos: listar, crear, editar, control de stock.
- Pedidos/comandas en tiempo real: añadir líneas de pedido, calcular total automáticamente.
- Notificaciones: avisos cuando los pedidos están listos.
- Factura y email: enviar facturas detalladas vía SMTP.
- Persistencia: Room + LiveData para datos en tiempo real, SharedPreferences para sesión y bar seleccionado.
- Modularidad: fragmentos para login, registro, dashboard, mesas y productos.
- UI/UX: diseño basado en prototipos Figma, planificación diagonal frontend/backend.

Actualmente MESERO APP no incluye una pasarela de pago, (Aplicación gratuita sin anuncios) y tampoco sincronización en la nube

Tecnologías utilizadas

En esta sección, vamos a desglosar las tecnologías, herramientas y patrones que se han utilizado en el desarrollo de la aplicación MESERO APP, así como su propósito dentro del proyecto.

Lenguaje de programación

- Java: Este es el lenguaje principal que hemos utilizado para desarrollar la aplicación en Android.

Entorno de desarrollo

- Android Studio: Es el IDE que hemos elegido para crear, ejecutar y depurar la aplicación.
- Warp: Esta terminal nos ayuda a gestionar comandos y controlar el proyecto.

Arquitectura y patrones

- MVVM (Model–View–ViewModel): Este patrón arquitectónico nos permite separar la lógica de negocio de la interfaz de usuario.
- LiveData: Lo utilizamos para actualizar la interfaz en tiempo real de manera automática.

Interfaz de usuario

- RecyclerView: Ideal para mostrar listas dinámicas.
- AlertDialog: Perfecto para mostrar mensajes y obtener confirmaciones.
- Fragments: Nos permiten tener una interfaz modular y reutilizable.

Persistencia de datos

- Room: Esta es nuestra base de datos local para almacenar la información de manera estructurada.
- SharedPreferences: Lo usamos para mantener la sesión y las configuraciones básicas.

Comunicación

- SMTP: Utilizamos este protocolo para enviar facturas por correo electrónico a través del servicio EmailSenderService.

Prototipado y control de versiones

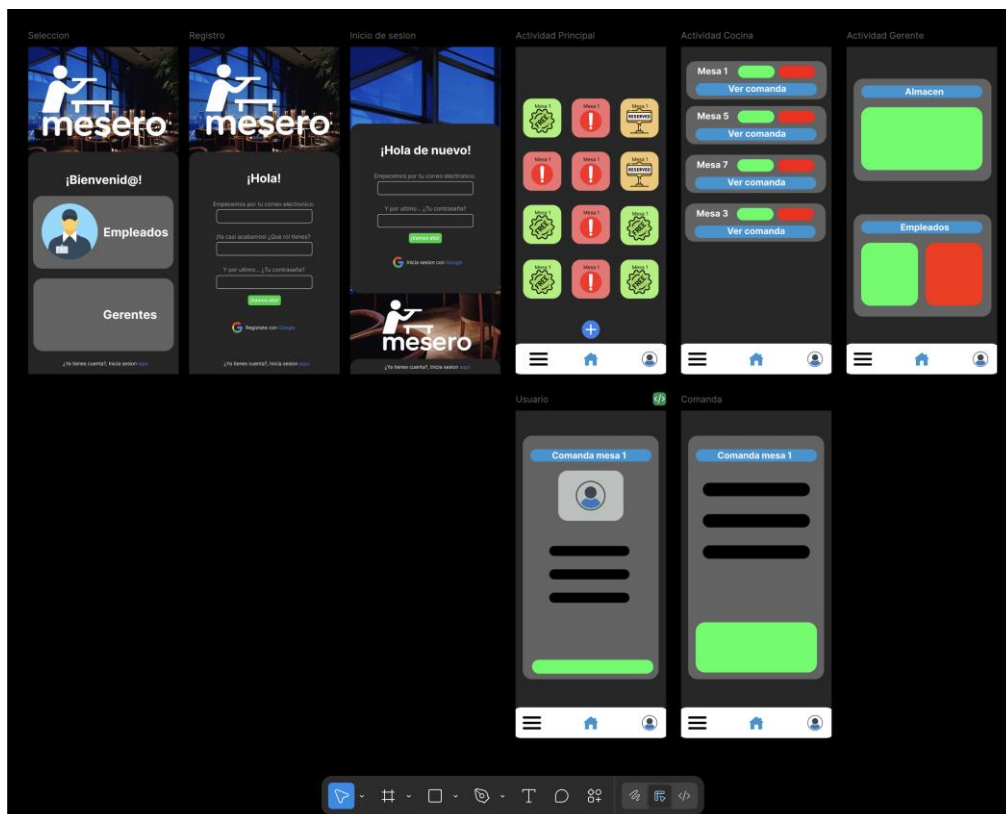
- Figma: Herramienta que utilizamos para crear prototipos visuales de la interfaz.
- GitHub: Nos ayuda a llevar el control de versiones del proyecto y a publicar el código fuente.

- README.md: Este archivo contiene la documentación principal del repositorio, creamos este con la siguiente herramienta readme.so. Esta herramienta permite previsualizar los estilos aplicados a el archivo.

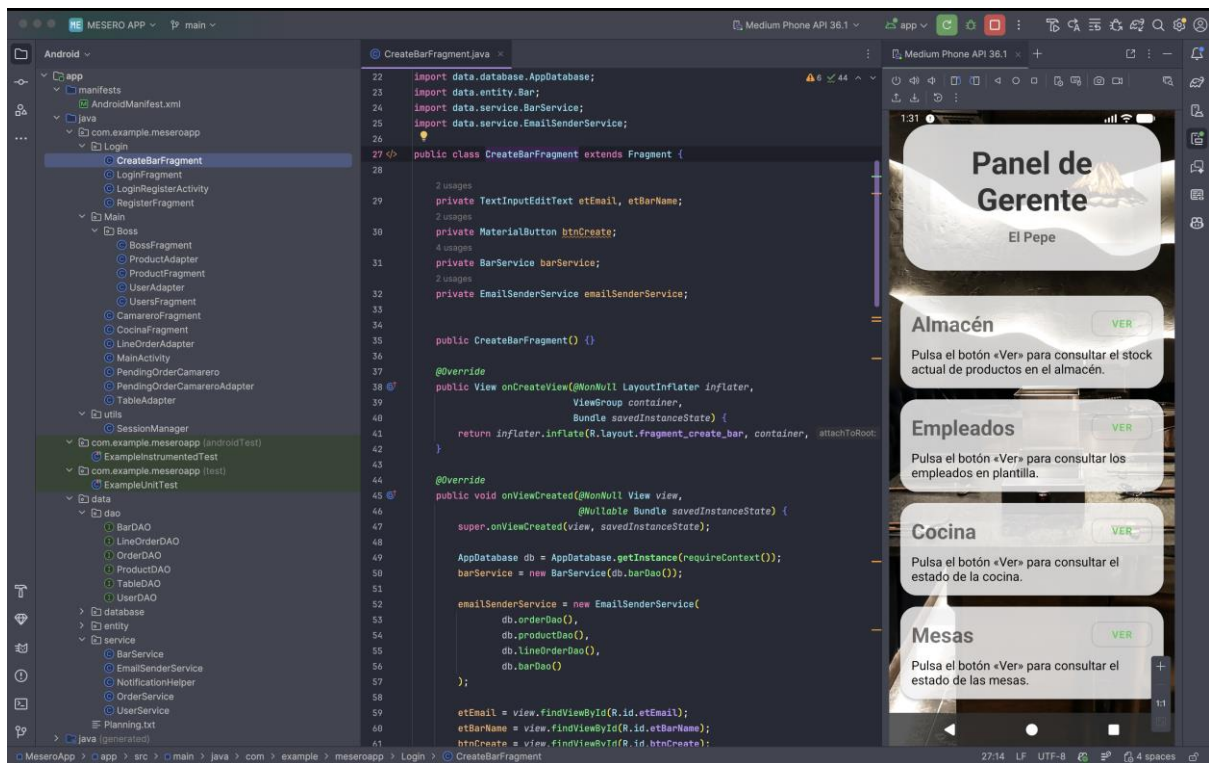
Evidencias de uso de tecnologías

A continuación, se presentan algunas imágenes que muestran cómo hemos utilizado varias de las tecnologías durante el desarrollo del proyecto:

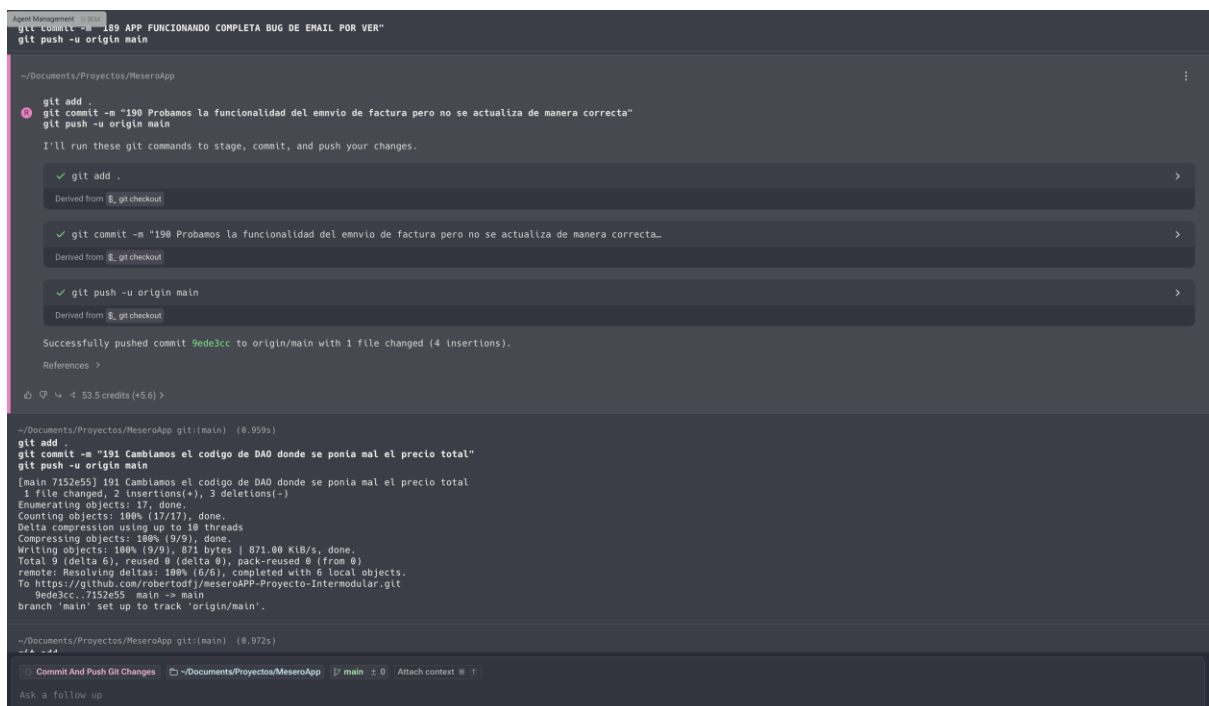
- Prototipado de la interfaz de usuario con Figma.



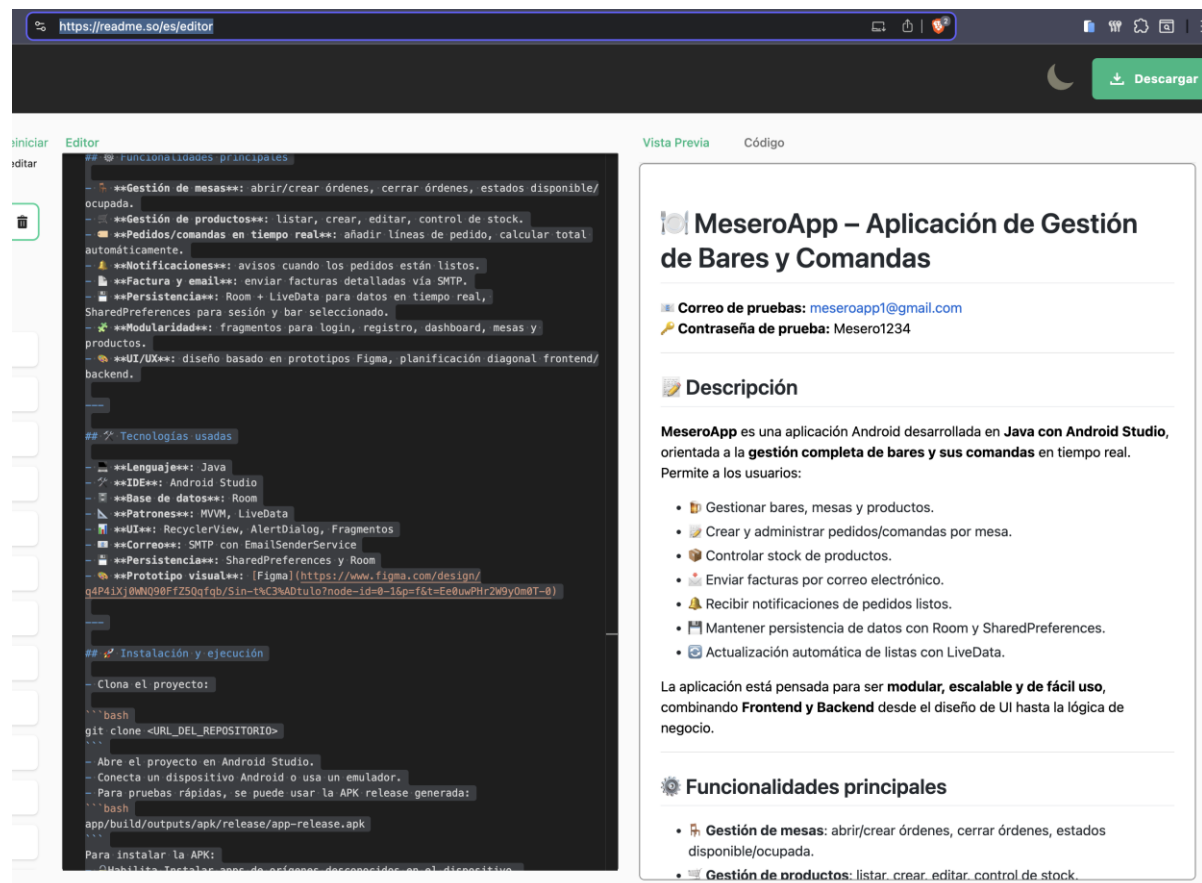
- Desarrollo y pruebas de la aplicación en Android Studio.



- Control de versiones utilizando Warp y GitHub.



- Creación del archivo README.md en el repositorio.



Análisis del sistema

Implementación

Explicar que hace cada parte del código

Manual de usuario

Explicar cómo realizar los registros / gestiones

Pruebas de caja blanca realizadas:

Prueba 1: Creación de bar con código incorrecto

Fragmento: BarFragment

Se probó el envío de un código incorrecto durante el proceso de creación de un bar.

El sistema respondió correctamente impidiendo la creación del bar, validando el funcionamiento del diálogo de entrada.

Prueba 2: Registro de usuarios con correo duplicado

Fragmento: RegisterFragment

Se detectó un error que permitía crear dos usuarios con el mismo correo electrónico.

Aunque la creación del usuario no se completaba, el sistema enviaba el token de verificación antes de realizar la comprobación del email.



Solución aplicada:

Se modificó el orden de validación para comprobar primero la existencia del correo electrónico antes de enviar el token, evitando así envíos innecesarios y una posible sobrecarga del servicio de correo.

```
Bar bar = db.barDao().getById(barId);
User userExist = db.userDao().getByEmail(email);
if (bar == null) {
    // Volvemos al UI para mostrar mensaje
    requireActivity().runOnUiThread() ->
        Toast.makeText(requireContext(), text: "No existe un bar
    );
    return; // salimos del hilo secundario
}
if (userExist != null){
    // Volvemos al UI para mostrar mensaje
    requireActivity().runOnUiThread() ->
        Toast.makeText(requireContext(), text: "El email ya esta
    );
    return; // salimos del hilo secundario
}
```

Resultado:

El sistema ahora bloquea correctamente el registro y no envía el token si el correo ya existe.

Prueba 3: Gestión de login

Se comprobó el correcto funcionamiento del inicio de sesión, verificando que el proceso se realiza sin errores.

Posteriormente, se detectó que el usuario no permanecía logueado al reiniciar la aplicación, lo que impedía gestionar notificaciones, estados activos y redirecciones según el rol.

Solución aplicada:

Se modificó el método de login para guardar correctamente el usuario en el SessionManager. Además, se añadió una comprobación desde la actividad principal para redirigir al login en caso de no existir una sesión activa.

```
tnLogin.setOnClickListener( View v -> {  
    String email = etEmail.getText().toString().trim();  
    String password = etPassword.getText().toString().trim();  
  
    new Thread() -> {  
        User user = userService.loginAndGetUser(email, password);  
  
        requireActivity().runOnUiThread() -> {  
            if (user != null) {  
                SessionManager.getInstance(requireContext())  
                    .saveSession(user.getId(), user.getBarId(), user.getRol());  
  
                Intent intent = new Intent(requireContext(), MainActivity.class);  
                startActivity(intent);  
                requireActivity().finish();  
            } else {  
                Toast.makeText(requireContext(),  
                    text: "Email o contraseña incorrectos o usuario inexistente",  
                    Toast.LENGTH_SHORT).show();  
            }  
        }  
    };  
}).start();
```

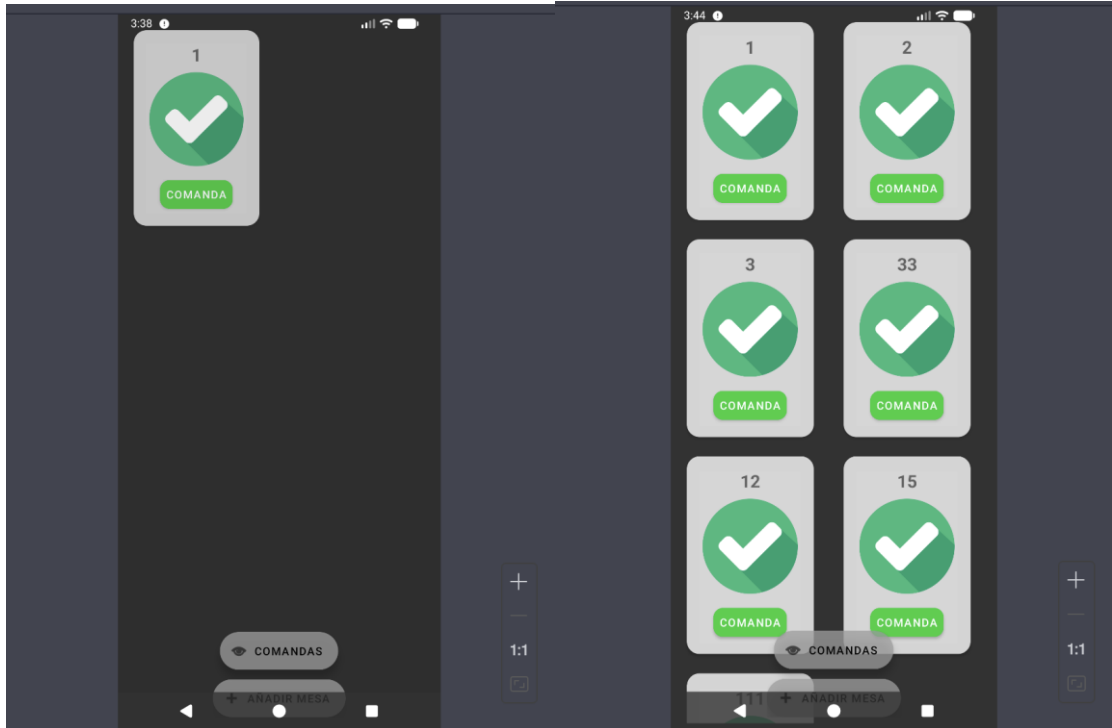
```
public User loginAndGetUser(String email, String password) {  
    User user = userDao.getByEmail(email);  
  
    if (user != null && user.getPassword().equals(password)) {  
        System.out.println("Login exitoso");  
        return user;  
    } else {  
        System.out.println("Email o contraseña incorrectos");  
        return null;  
    }  
}
```

Resultado:

La sesión se mantiene correctamente al volver a iniciar la aplicación.

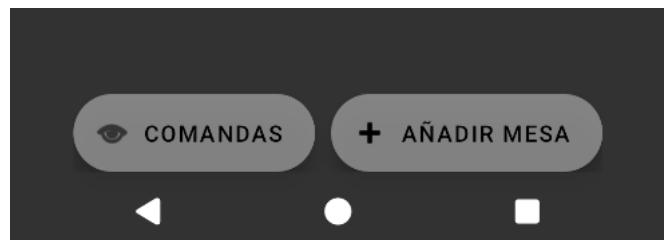
Prueba 4: Gestión de mesas y productos

Se verificó que las mesas se crean correctamente y que el funcionamiento del fragmento es adecuado, incluyendo el scroll y la reserva de mesas.



En caso de no existir productos asociados al bar, la aplicación no muestra productos, comportamiento esperado.

A nivel visual, se detectó que la colocación de los botones inferiores no era la más adecuada debido al margen, por lo que se ajustó el diseño para mejorar la usabilidad y cambiamos la colocación:



Prueba 5: Error al acceder a comandas

Al acceder al apartado de comandas, la aplicación se cerraba inesperadamente mostrando el siguiente error:

IllegalArgumentException: No view found for id com.example.meseroapp:id/fragment_container

Causa:

Se estaba intentando cargar un fragmento en un contenedor que no existía en el layout correspondiente.

```
private void viewPendingOrders() {
    PendingOrderCamarero pendingOrderCamarero = new PendingOrderCamarero();

    getParentFragmentManager() FragmentManager
        .beginTransaction() FragmentTransaction
        .replace(R.id.fragment_container, pendingOrderCamarero)
        .addToBackStack( name: null)
        .commit();
}
```

Solución aplicada:

Se corrigió el identificador del contenedor en el método replace().

```
private void viewPendingOrders() {
    PendingOrderCamarero pendingOrderCamarero = new PendingOrderCamarero();

    requireActivity() FragmentActivity
        .getSupportFragmentManager() FragmentManager
        .beginTransaction() FragmentTransaction
        .replace(R.id.fragment_containerMain, new PendingOrderCamarero())
        .addToBackStack( name: null)
        .commit();
}
```

Resultado:

El fragmento se carga correctamente sin provocar cierres inesperados.

Prueba 6: Navegación según rol de usuario

Se detectó que, al iniciar sesión como gerente, la aplicación no redirigía a la pantalla correspondiente debido a que el rol del usuario no se estaba guardando correctamente.

Solución aplicada:

Se corrigió el almacenamiento del usuario logueado y se ajustó la lógica de navegación según el rol.

```
13 > public class MainActivity extends AppCompatActivity {
14
15     @Override
16     protected void onCreate(Bundle savedInstanceState) {
17         super.onCreate(savedInstanceState);
18
19         if (!SessionManager.getInstance(context: this).isLoggedIn()) {
20             // Redirigir al login si no hay sesión
21             startActivity(new Intent(packageContext: this, LoginRegisterActivity.class));
22             finish();
23             return;
24         }
25
26         setContentView(R.layout.activity_main);
27
28         String role = SessionManager.getInstance(context: this).getUserRole();
29
30         switch (role) {
31             case "camarero":
32                 loadFragment(new CamareroFragment());
33                 break;
34
35             case "cocina":
36                 loadFragment(new CocinaFragment());
37                 break;
38
39             case "gerente":
40                 loadFragment(new BossFragment());
41                 break;
42
43             default:
44                 loadFragment(new CamareroFragment());
45                 break;
46         }
47     }
48
49     4 usages
50     private void loadFragment(Fragment fragment) {
51         getSupportFragmentManager().beginTransaction()
52             .replace(R.id.fragment_containerMain, fragment)
53             .commit();
54     }
```

Resultado:

La navegación funciona correctamente y se muestran únicamente las pestañas correspondientes.

Prueba 7: Visualización del código del bar

Se detectó que el código del bar mostraba siempre el valor 1, aunque internamente se aplicaba correctamente.

```
**Crear el bar aquí, solo si el token es correcto**
utors.newSingleThreadExecutor().execute(() -> {
    Bar bar = new Bar();
    bar.setBarName(barName);
    bar.setEmail(email);
    barService.createBar(bar);
    int barcode = bar.getId() + 1;

    new AlertDialog.Builder(requireContext())
        .setTitle("ATENCIÓN")
        .setMessage("Apunta este CÓDIGO BAR. Los empleados lo necesitarán: " + b
        .setPositiveButton(text: "Apuntado!", ( DialogInterface dialog2, int which2)
            RegisterFragment registerFragment = new RegisterFragment();
            getParentFragmentManager().beginTransaction()
                .replace(R.id.fragment_container, registerFragment)
                .commit();
        })
        .show();
});
```

Causa:

Room no actualizaba automáticamente el identificador mostrado.

```
public long createBar(Bar bar) {
    Bar existBar = barDao.getByBarName(bar.getBarName());

    if (existBar != null) {
        return -1; // Ya existe
    }

    return barDao.insert(bar);
}
```

Solución aplicada:

Se ajustó la obtención del dato para reflejar correctamente el código asignado.

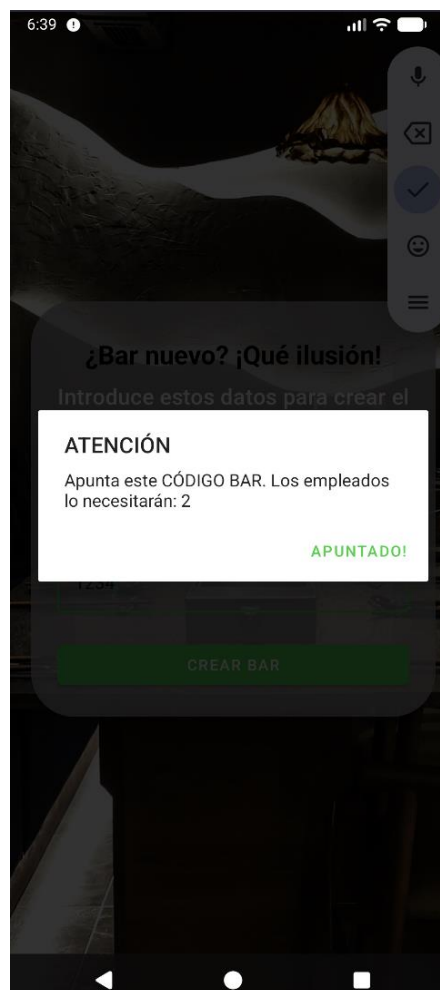
```
bar.setEmail(email);

long barId = barService.createBar(bar);

if (barId == -1) {
    requireActivity().runOnUiThread() ->
        new AlertDialog.Builder(requireContext())
            .setTitle("Error")
            .setMessage("No se pudo crear el bar. Inténtalo de nuevo.")
            .setPositiveButton(text: "OK", listener: null)
            .show()
    );
    return;
}
```

Resultado:

El código del bar se muestra correctamente a los usuarios.



Prueba 8: Error en el fragmento de cocina

Se produjo un cierre inesperado con el siguiente error:

FloatingActionButton cannot be cast to android.widget.Button

Solución aplicada:

Se corrigió el tipo de componente utilizado.

```
FloatingActionButton isActiveCocinero = view.findViewById(R.id.isActiveCocinero);
```

Resultado:

El fragmento de cocina funciona correctamente.

Prueba 9: Gestión de productos en almacén

Se añadió un botón para permitir la creación de productos en el almacén, aplicando validaciones de nombre y evitando valores nulos o vacíos.

Cada producto se asocia automáticamente al bar correspondiente.

```
public class ProductFragment extends Fragment {
    private void addProductBar() {
        dialog.setOnShowListener( DialogInterface d -> {
            dialog.getButton(AlertDialog.BUTTON_POSITIVE).setOnClickListener( View v
                new Thread() -> {
                    if (db.productDao().getByName(etName.getText().toString(), barId) != null) {
                        Toast.makeText(getContext(),
                            text: "Ese nombre de producto ya esta en uso, prueba otro",
                            Toast.LENGTH_SHORT).show();
                        return;
                    }

                    Product product = new Product();
                    product.setBarId(barId);
                    product.setProductName(etName.getText().toString());
                    product.setPrice(Double.parseDouble(etPrice.getText().toString()));
                    product.setStock(Integer.parseInt(etStock.getText().toString()));

                    db.productDao().insert(product);

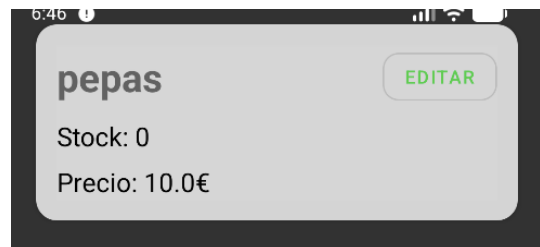
                    requireActivity().runOnUiThread() ->
                    Toast.makeText(getContext(),
                        text: "Producto añadido",
                        Toast.LENGTH_SHORT).show()
                );
            }).start();
        });
    }
}
```

Prueba 10: Flujo completo de pedidos

Se realizó una prueba completa del flujo de pedidos:

1. Apertura de orden.
2. Envío a cocina.
3. Descuento de stock.
4. Cierre de orden.

El funcionamiento fue correcto en todos los pasos.



Prueba 11: Acceso a base de datos en el hilo principal

Al marcar una comanda como realizada desde cocina, se produjo el siguiente error:

Cannot access database on the main thread since it may potentially lock the UI

Solución aplicada:

Las operaciones de base de datos se trasladaron a un hilo secundario.

```
builder.setPositiveButton( text: "Sí", ( DialogInterface dialog, int which) -> {  
    lineOrder.setCocinaDone(true);  
    lineOrder.setCamareroDone(false);  
    new Thread(() -> db.lineOrderDao().update(lineOrder)).start();  
  
    //Enviar notificación al camarero activo del bar  
    int barId = SessionManager.getInstance(getContext()).getBarId();  
    List<User> users = db.userDao().getActiveCamarero(barId);  
    if (users != null && !users.isEmpty()) {  
        for (User camarero : users) {  
            NotificationHelper.sendNotification(  
                requireContext(),  
                camarero.getId(),  
                title: "Pedido listo",  
                message: "Un pedido ha sido marcado como listo en la coc  
            );  
        }  
    }  
}
```

Resultado:

El error desaparece y la aplicación funciona con normalidad.

```
builder.setPositiveButton( text: "Sí", ( DialogInterface dialog, int which) -> {  
    lineOrder.setCocinaDone(true);  
    lineOrder.setCamareroDone(false);  
  
    new Thread(() -> {  
        db.lineOrderDao().update(lineOrder);  
  
        int barId = SessionManager.getInstance(getContext()).getBarId();  
        List<User> users = db.userDao().getActiveCamarero(barId);  
  
        if (users != null && !users.isEmpty()) {  
            for (User camarero : users) {  
                requireActivity().runOnUiThread(() -> {  
                    NotificationHelper.sendNotification(  
                        requireContext(),  
                        camarero.getId(),  
                        title: "Pedido listo",  
                        message: "Un pedido ha sido marcado como listo"  
                    );  
                })  
            }  
        }  
    }).start();  
});
```

Prueba 12: Cálculo del total en la factura

Se detectó un error en el cálculo del total de la factura, mostrando siempre un valor incorrecto.

 **MeseroApp** <meseroapp1@gmail.com>
para mí ▾

18:43 (hace 1 minuto) ☆ 😊 ↩ ⋮

Factura - El bar de prueba

Producto	Cantidad	Precio	Subtotal
1	1	1.0€	1.0€
pepas	10	10.0€	100.0€
Total			2.0€

Gracias por tu visita

Causa:

Error en la consulta DAO que asignaba el precio incorrectamente.

```
@Query("UPDATE `Order` SET totalPrice = :id")  
void updateTotalPrice(int id);
```

Solución aplicada:

Se corrigió la consulta SQL para calcular correctamente la suma de las líneas del pedido:

```
@Query("UPDATE `Order` SET totalPrice = (SELECT IFNULL(SUM(linePrice), 0) FROM  
LineOrder WHERE orderId = :orderId) WHERE id = :orderId")
```

Resultado:

El total de la factura se calcula correctamente.

**MeseroApp** <meseroapp1@gmail.com>
para mí ▾

14:57 (hace 0 minutos) ☆ 😊 ↩ ⋮

Factura - Prueba mesero

Producto	Cantidad	Precio	Subtotal
Merluza a la plancha	2	12.0€	24.0€
Cerveza Estrella	2	1.5€	3.0€
Total			27.0€

Gracias por tu visita

↩ Responder ↪ Reenviar 😊